Task Tracker

Marius's deliverable is a reading-and-scoping writeup covering everything you took notes on. Two items inside it are the ones he weighted: **find the real-world target** and **refine the plan**. The other six are the reading that feeds those two. Do the six reading tasks first, let them point at a target, then lock the plan.

Status key: Not started, In progress, Done.

#	Task	Priority	Feeds	Dependency	Status
T1	**Find the real-world target.** A C/C++ Android component with a recent CVE, that builds with Scudo and runs under MTE. Assess exploitability under the hardened allocator.	**Highest**	Plan's final phase	T2, T4, T6 inform it	**Done** — CVE-2025-48592 (dav1d), see Target tab
T2	**Refine the plan** with clear execution steps.	**Highest**	The deliverable	Needs T1's target	In progress — see Plan tab
T3	**Project Zero review.** MTE-as-implemented series + Pixel chains; open vulnerabilities.	High	T1, lit section	None	**Mostly done** — see Project Zero tab; Pt 1 remaining
T4	**Scudo + MTE/PAC notes.** How Scudo invokes MTE; where PA enters.	High	T1, audit phase	None	**Done** — source read, see Scudo & MTE tab
T5	**Research-paper review.** How platforms integrate MTE/PA; open vulnerabilities.	Medium	Lit section	None	Mostly done — IUBIK + 3 attack papers + PZ series
T6	**PA / Scudo-MTE opt-in mechanics.** How an app opts into MTE memory; enabling PA.	Medium	T1, harness	Overlaps T4	Partly done — opt-in path in Scudo & MTE tab; PA-enable open
T7	**BTI note (reference only).** ARM's analogue to Intel CET IBT.	Low	Background	None	**Done** — see BTI tab

#	Task	Priority	Feeds	Dependency	Status
T8	**Confirm scope questions** (Scudo revision, SYNC/ASYNC, success criterion, Berkeley API keys).	Medium	Everything	Needs Marius	Not started

How to sequence the week

1. **Days 1–2:** finish T3 and T4 (done); add T7.
2. **Days 2–4:** T1 target scouting (done — dav1d lead).
3. **Days 4–5:** T5 and T6 to fill gaps.
4. **Day 5:** lock the plan around the chosen target; send T8's questions to Marius.

Answers to Marius's questions

Short versions, for talking through. Full detail lives in the deliverable tabs.

1. Latest Project Zero findings — what do they say about vulnerabilities?

Project Zero's "MTE As Implemented" series (Mark Brand) is the key read. Their bottom line: **MTE is not a reliable hard wall — it's a speed bump whose height depends on context.** They sort every bypass into two buckets:

- **Known-tag:** learn the tag (usually via a speculative/timing side channel), then your bad access carries the right tag and never faults. Wherever an attacker can build such a side channel, MTE's probabilistic protection fundamentally fails.
- **Unknown-tag:** succeed even with the wrong tag, by exploiting an implementation gap (async window, untagged memory, etc.).

They also rank where MTE actually bites: **weak** against a local attacker who can run code (Chrome renderer), **strong** against a remote attacker hitting a media/file parser (a messaging app) — because that attacker can't easily build the side channel. The Pixel 10 post adds a real example: a recent exploit chain only needed small tweaks to survive new hardware mitigations (PAC cost them one extra step, not a wall).

2. Open vulnerabilities that could hypothetically be exploited (MTE/PA)

Ways to skip the 1-in-15 guess:

- **Side channels** that leak the tag without crashing (TikTag, StickyTags for MTE; PACMAN for PA). The big one.
- **Untagged memory** — globals/ `.data` have no tag to mismatch; Android's zygote sharing makes pointer-forging across apps easy.
- **Intra-granule overflow** — a small overflow staying inside one 16-byte chunk shares the tag, so it's invisible.
- **Check suppression** — code inside Scudo's `ScopedDisableMemoryTagChecks` (PSTATE.TCO) runs with checks off; the question is whether attacker data reaches it.
- **Async-mode window** — the bad access is observable before the delayed fault lands.
- **Tag exclusions** — tag 0 excluded in Scudo (userspace); tag 15 bypasses checks in the kernel.
- **Repeated attempts** against an app that respawns without re-randomizing tags.

PA-specific: short signatures (brute-forceable with an oracle), substitution (swap one validly-signed pointer for another), signing/auth oracles, and data-only attacks PA never covers.

3. How Scudo works, and how it uses MTE

Scudo is Android's default `malloc` / `free` implementation (since Android 11). A hardened allocator: fast size-class pools (primary), guard-paged large allocations (secondary), an 8-byte header with a 16-bit CRC checksum (not cryptographic, not a PAC), randomized placement, optional quarantine (off by default).

How it uses MTE: MTE is the hardware mechanism — the CPU stores a 4-bit tag per 16-byte granule and checks it on every load/store, but it has no opinion on *which* tags to use. **Scudo is the one that implements the policy on top of that mechanism:** on each allocation it picks a random non-zero tag (`IRG`), stamps it across the allocation while zeroing it (`STZG` / `DC GZVA`), and returns the tagged pointer; on `free` it retags so stale pointers mismatch. Lock vs locksmith: MTE is the lock, Scudo assigns and changes the keys. (Confirmed from source: Scudo's tagging is MTE-only — no PA in it.)

4. Our real-world target

Lead: dav1d, CVE-2025-48592. A heap buffer overflow → out-of-bounds read in `C2SoftDav1dDec.cpp`, Android's AV1 video-decoder wrapper. Why it fits: it's C/C++ on ARM, runs on Scudo, is remote and zero-click (a crafted video triggers it, no user tap), affects Android 15/16, and has a patch commit to diff. It sits in the remote-parser context where Project Zero says MTE is *strongest*, which makes "can an agent beat it anyway" a genuinely open question.

Exploitability under the hardened allocator: it's a *read* (info leak). With MTE SYNC on, the overread crosses into a differently-tagged neighbor and **crashes ~15/16 of the time** — so it most likely degrades from a leak to a crash, *unless* the overread stays within one 16-byte granule, or the attacker can learn the tag (hard, remotely). That "MTE probably wins here" is itself a publishable result.

Backups: libaom (CVE-2024-5171, integer→heap overflow, another clean standalone C library) and Android Bluetooth SDP use-after-frees (CVE-2025-0084/0075/0074) — UAFs test *temporal* detection and give a clearer write-style "security effect" than an OOB read. Not better than dav1d overall, but worth having a UAF in the mix.

5. The plan, step by step (simplified)

The one question we're answering: **how good is an LLM agent at finding and exploiting memory-safety bugs — and does a hardened allocator (Scudo + MTE) stop it?** We start tiny and build up, so every result is honest.

1. **Prove the ruler works.** Write a few deliberately-buggy C programs; confirm MTE catches what it should and lets valid runs through. Build the success-checker and logging *before* any agent.
2. **Measure the baselines.** Run a dumb random-guesser and a normal fuzzer on those programs. Now we have numbers to beat.
3. **Bring in the agent.** Same programs, same rules; vary what it's given (source, binary, crash logs). Compare to the baselines. Classify *why* it wins when it wins.
4. **Reproduce a known attack.** Can the agent redo a published MTE bypass? (Kept separate from new tasks, since it may be in training data.)
5. **The real target.** Point it at dav1d/CVE-2025-48592 under Scudo + MTE and reason + measure: does the bug survive the hardened allocator, and can the agent make it?

Throughout: an independent checker decides success, we replay wins from clean state, and we report costs and failures too — not just the best run.

6. Two targets: a read and a write (decision to confirm with Marius)

We should use **two** real-world targets, one of each bug shape, because they stress different halves of what MTE does. State both drawbacks openly — that honesty is part of the contribution.

- **Read target — dav1d (CVE-2025-48592).** An out-of-bounds *read* (info leak). Tests whether MTE catches a bad read (spatial detection). *Drawback:* "success" is weak — a leak, and MTE most likely just turns it into a crash. It mostly measures MTE at its **strongest** (remote parser).
- **Write target — Skia (CVE-2025-32318).** An out-of-bounds *write* → privilege escalation. Tests the corruption-to-real-effect path MTE is actually built to stop, so "success" is a clear security win. *Drawbacks:* it's privilege-escalation-flavored, which sits nearer MTE's **weaker** end, and Skia is a big codebase that's much harder to build and isolate than one small decoder.

Sequencing (important): do **dav1d first** — it's one self-contained decoder, so it's the fastest way to get the harness and success-checker working end to end. Only *after* that machinery is proven, add Skia for the write case. Don't start both in parallel; that's how the project stalls.

Ask Marius: one target done deeply for a first result, or commit to both now? Bring it as a choice, not a done deal — it's a scope call that's his to make. Also ask: how is the Skia bug *reached* — remote-rendered content, or only after a local foothold? That answer decides whether Skia tests MTE at its strong or weak end.